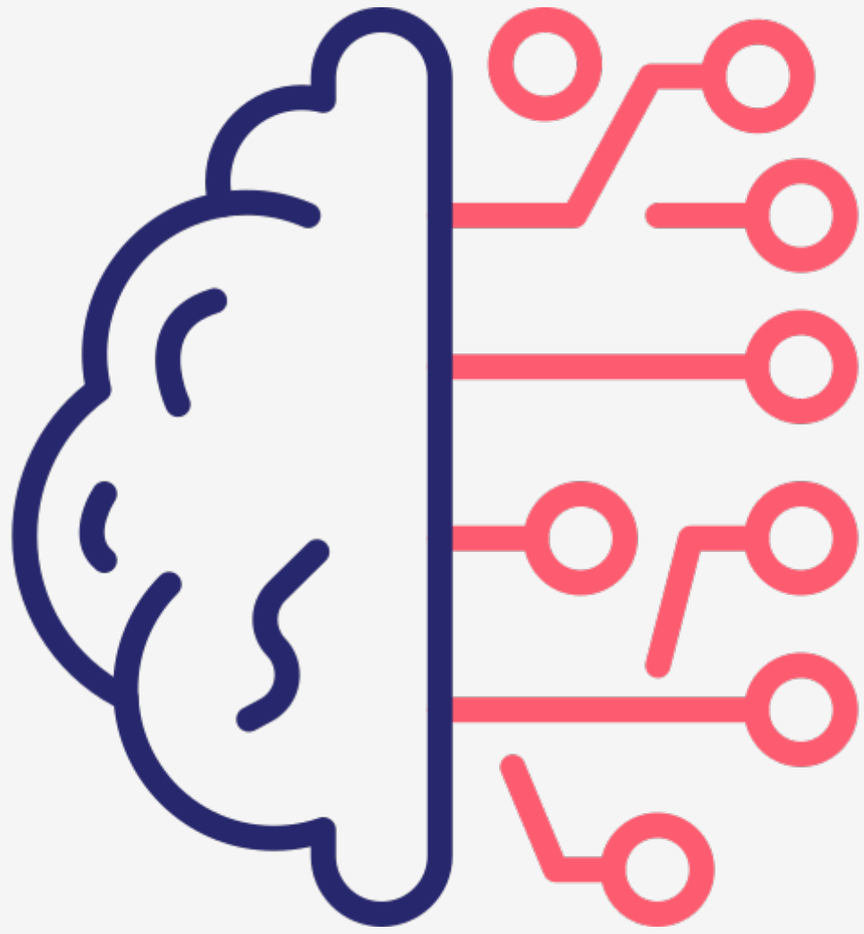


What Makes an LLM “Large”?



An LLM is an **Ai model Trained on Massive amounts** of ***Text*** so it can understand & generate ***Human-like*** language.

What makes an **LLM** “Large”?

Large Language Model has Millions to **Billions** of **Parameters** (**internal weights**).

Examples:

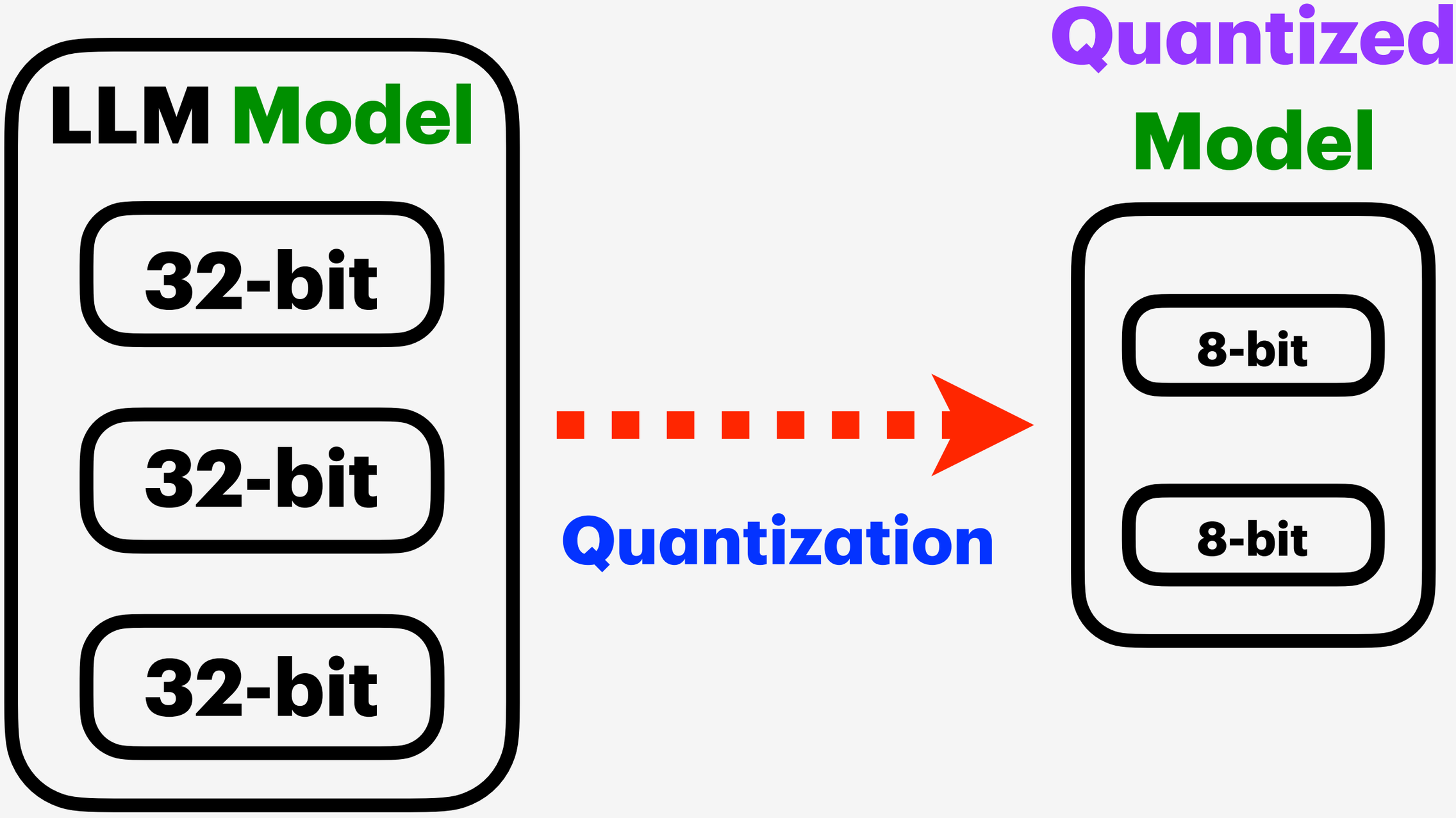
LLaMA 3 → 8B, 70B **GPT** → 7B to 175B+ **Mistral** → 7B, 12B

Qwen → 7B, 72B

More Parameters = More Knowledge + Better Reasoning (in General).

Quantization

Quantization is the process of converting an **LLM model's** high-precision numerical values **(like 32-bit or 16-bit floats)** into lower-precision formats (like **8-bit** or **4-bit integers**) to reduce **memory** usage and speed up **inference**, with minimal accuracy loss.



Quantized LLM is the lightweight version of a Heavy Model

A **Quantized LLM** is basically a compressed version of a large language model where the internal numbers (weights & activations) are stored using fewer bits instead of the usual **16-bit** or **32-bit** floating-point formats.

Analogy:

Original Model = high-resolution RAW image

Quantized Model = compressed **JPEG**

High-precision values (FP32/FP16):	Quantized lower-precision values:
0.512345	0.51 (FP16)
-0.0038921	0 (INT8)
12.982374	13 (INT8)
-4.120497	-4 (INT8)

Why Quantize?

Because full-precision models are **Bulky**.

For Example, A 70B parameter model in FP16 can be ~140 GB.

After quantizing (say 4-bit), that same model might drop to ~35 GB.

How it Works?

A **model** basically stores a **TON** of numbers ("**weights**").

Quantization turns those **numbers** into: **INT8** **INT4**

Even **INT2** in some experimental setups

Types of Quantization

8-bit (INT8) → widely used, almost no noticeable quality drop

4-bit (INT4) → sweet spot for running big models locally (GGUF, GPTQ, AWQ)

3-bit / 2-bit → research / niche; starts to hurt model quality

Real-life Usage

LLMs on laptops (LLaMA, Mistral, Qwen in GGUF)

Deploying large models cheaply on AWS / GCP

Faster inference for chatbots / API workloads